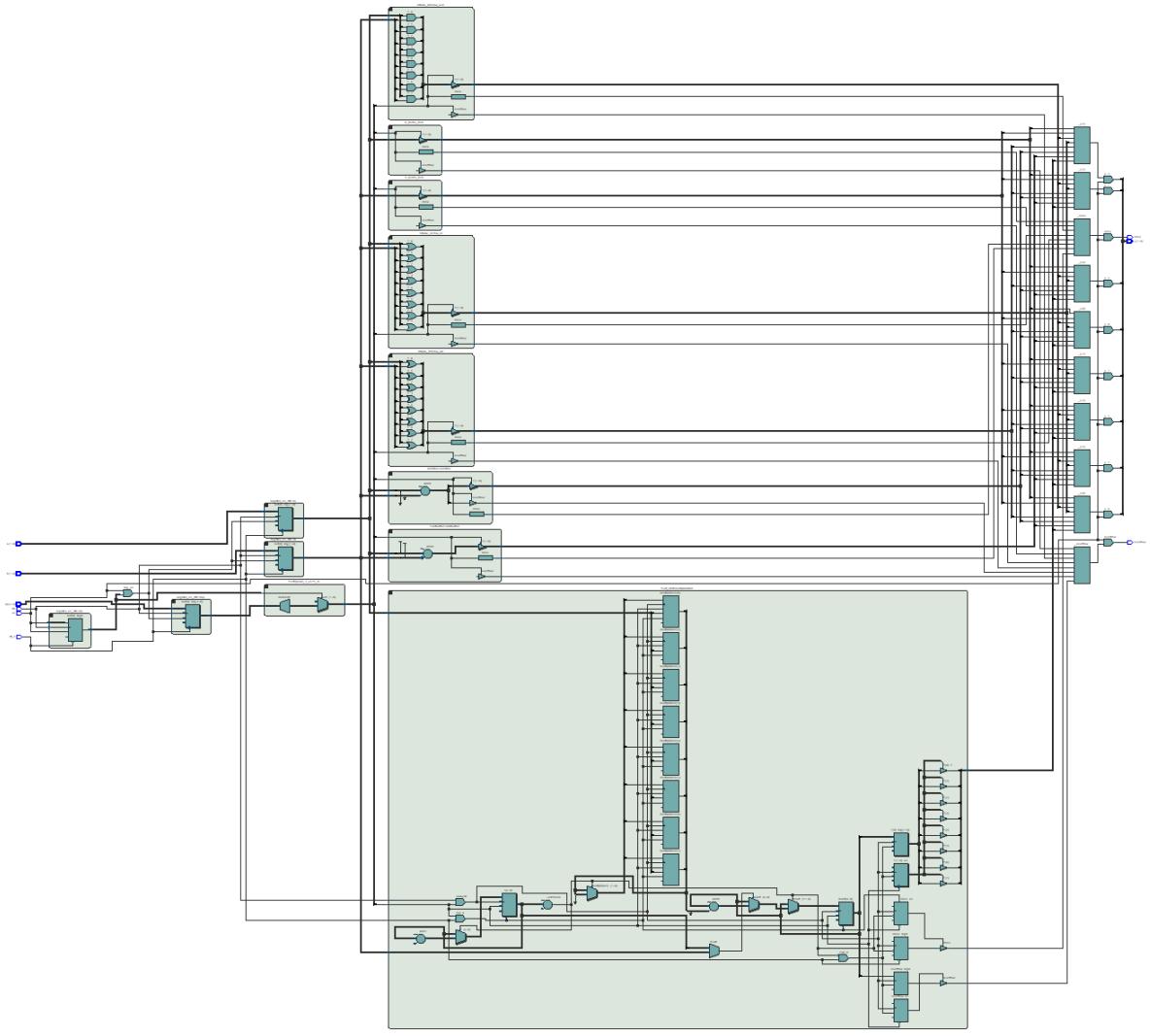
ECE 204 Design Project

8 Bit ALU

Nolan Kessler (kesslnol@oregonstate.edu)
Maxwell Fabian (fabianma@oregonstate.edu)
John O'Connor (oconnoj5@oregonstate.edu)

June 7, 2026

Demonstration Video:
https://media.oregonstate.edu/media/t/1_ci3vfnca



Contents

1 8 Bit ALU Functional Description

This 8 bit ALU accepts two 8 bit operands A and B, along with a 3 bit opcode (OP). The state of the ALU is controlled by an active high enable pin (EN), and can be reset using the active low reset pin (RST_N). As this ALU supports sequential operations which may take more than one clock cycle, the CLK input must be connected to a clock signal. The ALU provides one 8 bit result output (Y) an overflow flag (OVER) and a done flag (DONE). A table of the ALU's supported operations is given in table 1. A diagram of the inputs and outputs is shown in figure 1. A description of the function of each bus/pin is given in table 2.

Operation	**Opcode**	**Clock Cycles**	**Overflow Condition**	**Description**
A pass	0x0	1	N/A	Passes A to Y unmodified
B pass	0x1	1	N/A	Passes B to Y unmodified
Bitwise AND	0x2	1	N/A	Y is bitwise AND combination of operands A and B
Bitwise OR	0x3	1	N/A	Y is bitwise OR combination of operands A and B
Bitwise XOR	0x4	1	N/A	Y is bitwise XOR combination of operands A and B
Addition	0x5	1	$A + B > 255$	Y is the sum of operands A and B
Subtraction	0x6	1	$A + (\bar{B} + 1) > 255$	Y is the sum of A and the 2's complement inverse of B
Multiplication	0x7	9	$A * B > 255$	Y is the product of A and B

Table 1: Table of supported operations

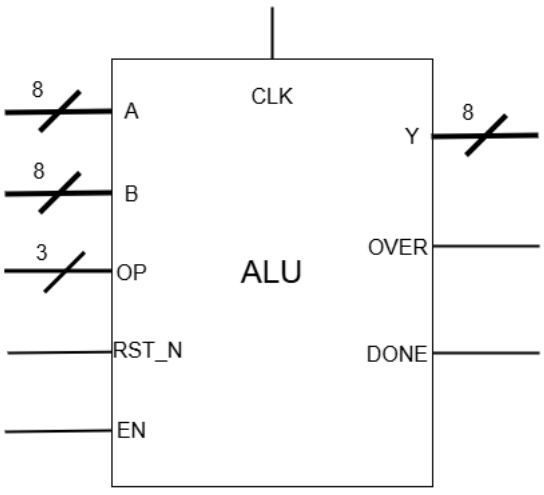


Figure 1: ALU Inputs and Outputs

Bus / Pin	Width (Bits)	Function
Input		
A	8	First 8-Bit operand - input to mathematical or logical operation.
B	8	Second 8-Bit operand - input to mathematical or logical operation.
OP	3	Opcode - selects which operation to perform.
RST_N	1	Active low asynchronous reset - puts the ALU into initial ready state.
EN	1	Enable - starts ALU when high if in ready state. Must be high for ALU to output data.
Output		
Y	8	8 Bit result - output of the ALU operation
OVER	1	Overflow flag - high value generally indicates result cannot be represented in bits of Y. Exact meaning can vary between operations.
DONE	1	Is low while operation is in progress or ALU is ready, high when operation is completed.

Table 2: ALU IO Functional Descriptions

1.1 ALU Operation

Before beginning an operation the ALU must be reset by setting RST_N low, after reset occurs the ALU is considered to be in the "ready" state. EN must be low for all rising edges of CLK while the ALU is in the ready state. Once the inputs on A, B, and OP are stable, EN can be set high to begin ALU operation. The rising edge of EN must occur when CLK is low. EN must remain high until after the next falling edge of CLK. At this point the ALU enters the "running" state. From this point on the value of EN can be changed without effecting the ALU (or the result when the ALU is done). The ALU will always output all zeros when EN is low. When DONE goes high the ALU is in the "done" state. The operation is finished, and results can be read from Y, and OVER. A timing diagram of a normal ALU operation is shown in figure 2.

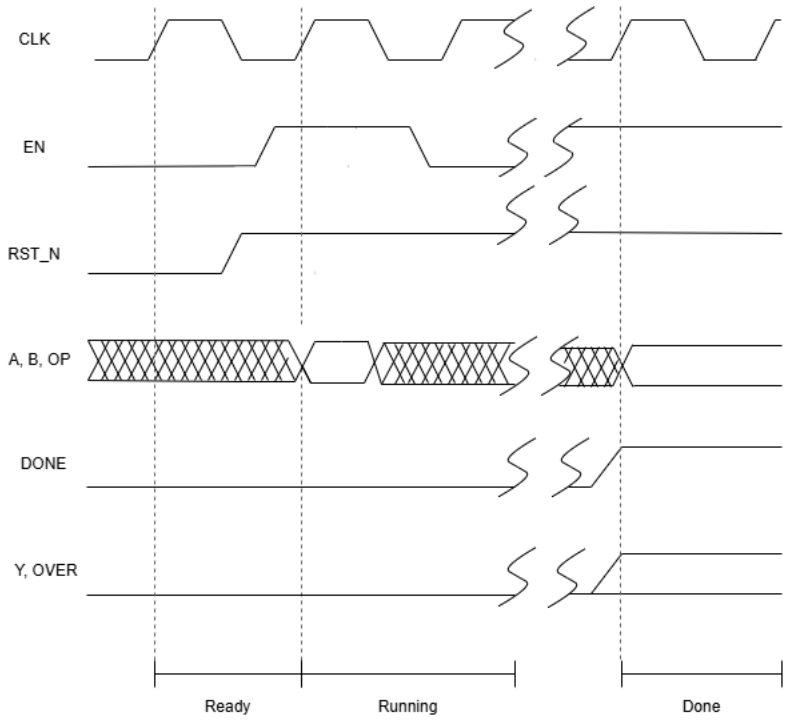


Figure 2: ALU Operation Timing Diagram

2 Design and Implementation

This ALU is designed to utilize modular operations, to allow the design to allow for easy testing during the design process, and potential rapid addition of new functions in the future. These modular operations are then linked together with integrated together with structures to allow for the selection of operations, forming the complete ALU.

2.1 Operation Modules

Each operation module accepts the inputs, and supplies the outputs shown in table 3. All outputs of the modules are routed through tri-state buffers - the outputs of an operation are floating unless its enable pin is high.

There are two primary types of operation modules utilized in this ALU, combinational logic operations, and sequential logic operations. For combinational logic operations, the CLK and RST_N inputs are not needed, and DONE is always high when the module is enabled. A diagram of such an operation module is shown in figure 3. Sequential logic operations utilize all inputs to perform operations that require more than one clock cycle. The only such operation in this ALU is multiplication. A diagram of this type of module is shown in 4.

The following sections will include a brief description of the implementation of each ALU operation, and a listing of its SystemVerilog code.

2.1.1 A Pass

A Passthrough passes the 8-bit operand A to the output Y without modification. In the case enable is low, all outputs float to Z.

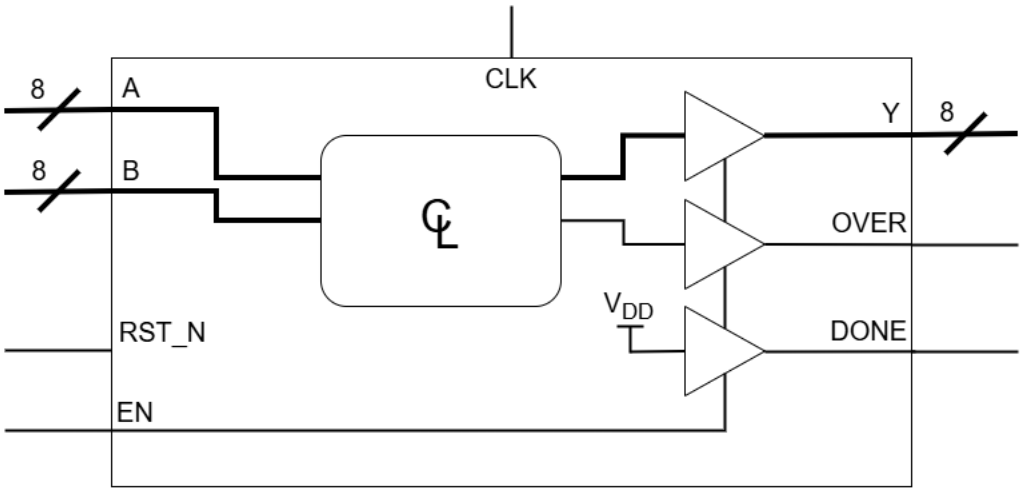


Figure 3: Schematic of a Combinational Operation Module

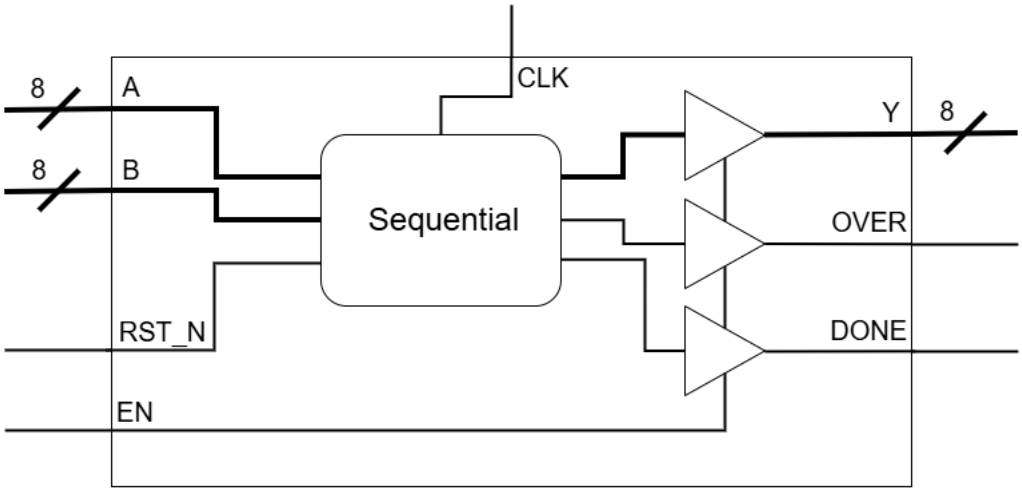


Figure 4: Schematic of a Sequential Operation Module

Listing: a_pass.sv

```
/*
Project: 8-Bit ALU
Author: John O'Connor
Sources: N/A

Module for A passthrough
*/

module A_pass(
        input logic [7:0] A,
        input logic enable,
        output logic [7:0] Y,
        output logic done,
        output logic overflow

);
```

Bus / Pin	Width (Bits)	Function
Input		
A	8	First 8-Bit operand - input to operation.
B	8	Second 8-Bit operand - input to operation.
RST_N	1	Active low asynchronous reset - resets the module to inial ready state.
EN	1	Enable - enables the module. The module will only output data when this pin is high
Output		
Y	8	8 Bit result - output of the operation
OVER	1	Overflow flag - high value generally indicates result cannot be represented in bits of Y. Exact meaning can vary between operations.
DONE	1	Is low while operation is in progress or is ready, high when operation is completed.

Table 3: Operation Module IO Descriptions

```
17
18          assign Y = enable ? A : 8'bz; //Assigns Y when enable is high
19          assign done = enable ? 1'b1 : 1'bz;  // Done is high when enable is high
20          assign overflow = enable ? 1'b0 : 1'bz;  // No overflow occurs for passthrough
21
22
23
24  endmodule
```

2.1.2 B Pass

B Passthrough passes the 8-bit operand B to the output Y without modification. In the case enable is low, all outputs float to Z.

Listing: b_pass.sv

```
1   /*
2   Project: 8-Bit ALU
3   Author: John O'Connor
4   Sources: N/A
5
6   Module for B passthrough
7   */
8
9   module B_pass(
10          input logic [7:0] B,
11          input logic enable,
```

```
        output logic [7:0] Y,
        output logic done,
        output logic overflow
);

        assign Y = enable ? B : 8'bz;  //Assigns Y when enable is high
        assign done = enable ? 1'b1 : 1'bz;  // Done is high when enable is high
    assign overflow = enable ? 1'b0 : 1'bz;  // No overflow occurs for passthrough


endmodule
```

2.1.3 Bitwise AND

Bitwise AND performs the logical AND operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if and only if the corresponding bits and A and B are booth 1. When enable is low, all outputs float.

Listing: bw_and.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources: N/A

Module for running the Bitwise AND in the ALU
*/
module Bitwise_AND(
    input logic [7:0] A, B, //8-bit Operand inputs
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   //8-bit Output
    output logic done,      //Done Flag
    output logic overflow   //Overflow Flag
);

    //Assigns Y to the output of Bitwise AND when enable is high
    assign Y = enable ? (A & B)  : 8'bz;

    //Simple Functions like this one have simple done and overflow flags
    assign done     = enable ? 1'b1     : 1'bz;
    assign overflow = enable ? 1'b0     : 1'bz;

endmodule
```

2.1.4 Bitwise OR

Bitwise OR performs the logical OR operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if either or both corresponding bits in A and B are 1. When enable is low, all outputs float.

Listing: bw_or.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources: N/A

Module for running the Bitwise OR in the ALU
*/
module Bitwise_OR (
    input logic [7:0] A, B, //8-bit Operand inputs
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   //8-bit Output
    output logic done,      //Done Flag
    output logic overflow   //Overflow Flag
);

    //Assigns Y to the output of Bitwise OR when enable is high
    assign Y = enable ? (A | B) : 8'bz;

    //Simple Functions like this one have simple done and overflow flags
    assign done     = enable ? 1'b1:1'bz;
    assign overflow = enable ? 1'b0:1'bz;

endmodule
```

2.1.5 Bitwise XOR

Bitwise XOR performs the logical XOR operation on each corresponding bit of the operands A and B, setting the result of this to Y. A bit in Y is 1 if either the of the corresponding bits in A or B is 1 but both. When enable is low, all outputs float.

Listing: bw_xor.sv

```
/*
Project: 8-Bit ALU
Author: John O'Connor
Sources: N/A

Module for Bitwise XOR.
*/

module Bitwise_XOR (
        input logic [7:0] A, B,
        input logic enable,
        output logic [7:0] Y,
        output logic done,
        output logic overflow
);


        assign Y = enable ? (A ^ B) : 8'bz;  // Assigns Y when enable is high, otherwise Y is
        ↪  floating
```

```
        assign done = enable ? 1'b1 : 1'bz;  // Done is high when enable is high
    assign overflow = enable ? 1'b0 : 1'bz;  // No overflow occurs with bitwise XOR

endmodule
```

2.1.6 Addition

Addition performs unsigned addition on two 8-bit operands A and B, setting the result of this to Y. In the case where the result of A + B exceeds 255, overflow will be set to 1, representing the most significant bit. When enable is low, all outputs float.

Listing: addition.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources: N/A

Module for running the adder in the ALU
*/
module Addition(
    input logic [7:0] A, B, //8-bit operand inputs
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   //8-bit output
    output logic overflow,  //Overflow Flag
    output logic done       //Done Flag
);
    //Assigns overflow as 9th bit to Y and takes adder when enable is high
    assign {overflow, Y} = enable ? (A + B) : 9'bz;

    //Done flag on enable high
    assign done = enable ? 1'b1:1'bz;

endmodule
```

2.1.7 Subtraction

Subtraction performs unsigned subtraction on two 8-bit operands A and B, setting the result of this to Y. This implementation uses the two's compliment of B and computes subtraction by adding A to the two's compliment of B. When enable is low, all outputs float.

Listing: subtraction.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources: N/A

Module for running the Subtractor in the ALU
```

```
*/
module Subtraction (
    input logic [7:0] A, B, //8-bit Operand inputs
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   //8-bit Output
    output logic done,      //Done Flag
    output logic overflow   //Overflow Flag
);

    //Assigns Y to the output of a subtractor when enable is high
    assign Y = enable ? (A - B) : 8'bz;

    //Simple Functions like this one have simple done and overflow flags
    assign done     = enable ? 1'b1:1'bz;
    assign overflow = enable ? 1'b0:1'bz;

endmodule
```

2.1.8 Multiplication

Multiplication performs a binary sequential multiplication by running for 9 clock cycles for 8 cycles the value for A has a logical left shift applied and is added to the result register if the equivalent bit in B is high. This functions similarly to writing out long multiplication by hand. The schematic is shown in Figure 5. This can also be modeled as a state machine with 3 states, an initial state,

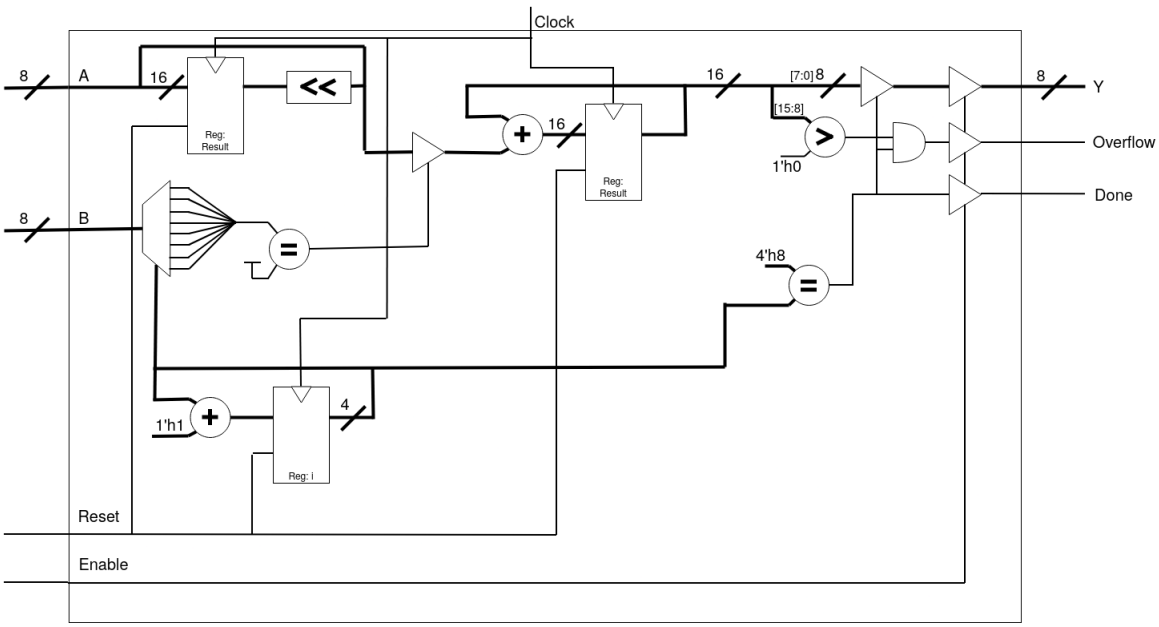


Figure 5: Schematic of the Multiplication Module

multiplication state, and an finished state. Where the initial state sets the registers to their reset state, sending to the multiplication state. The multiplication state shifts the multiplicand and adds it to the result when conditions are met, and re-enters the multiplication state until it has completed this state 8 times, exiting to the final state. The final state sets the done flag high and sets the output and overflow accordingly. This can be visualized in Figure 6.

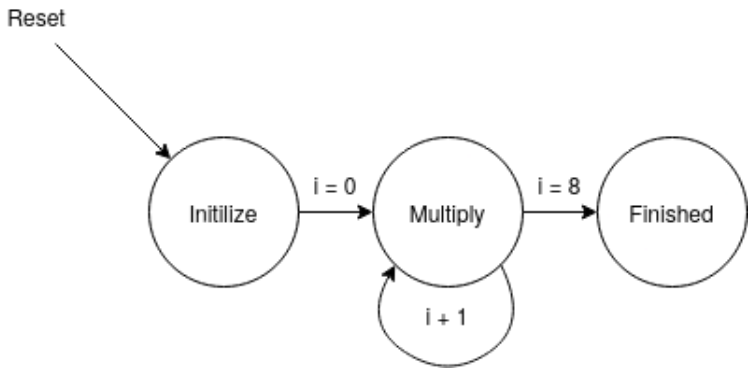


Figure 6: State Machine Diagram for the Multiplication Module

Listing: mult.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources:
    *https://en.wikipedia.org/wiki/Binary_multiplier#Single-cycle_multiplier
    ↪ *https://stackoverflow.com/questions/63655634/8-bit-sequential-multiplier-using-add-and-shift

Sequential Multiplier, functions similarly to doing long multiplication by hand
*/
module Mult_8bit (
    input logic [7:0] A, B, //8-bit Operand inputs
    input logic clock,      //clock to allow for sequential logic
    input logic reset_n,    //Resets internal registers to base state
    input logic enable,     //Enable - sets output of operator to floating
    output logic [7:0] Y,   // 8-bit Output
    output logic done,      //Done Flag
    output logic overflow   //Overflow Flag
);

//Registers for helping track the current sequence in the multiplication
logic [15:0] result;
logic [15:0] multiplicand;
logic [3:0]  i;

//Wire for the highest 8 bits of result, to help deconstruct overflow
logic [7:0]  cout;

    //Needs to trigger only when enable is on or when reseting
    always @(posedge (clock && enable) or negedge reset_n or negedge(enable)) begin
        //Resets all the registers and the done flag to initial states
        if(~reset_n) begin
            result          <= 0;
            multiplicand    <= A;
            i               <= 0;

            //Functions as equivalent to enable to them to function reasonably
            // asynchronus
```

```
            Y        <= 8'bz;
            done     <= 1'bz;
            overflow <= 1'bz;
        end
        //Sets the outputs to floating when the enable is off
        else if(~enable)begin
            Y        <= 8'bz;
            done     <= 1'bz;
            overflow <= 1'bz;
        end else begin
            //For loop created using a counter and comparitor
            //Functionally same as:
            //  for(int i = 0; i < 8; i++)
            //  counts from 0 to 7 inclusively
            if(i < 8) begin
                //Performed on every iteration so the current
                // sequence in multiplication is correct
                multiplicand <= multiplicand << 1; //Logical Left Shift
                                                   //I.E. multiplied by 2

                //Reads each bit of B and increases the result by the
                // current left shifted multiplicand
                if(B[i] == 1) begin
                    result <= result + multiplicand;
                end
            //Iterand counting by one to complete for loop functionality
            i <= i + 1;

            //Once the loop has finished the result can be thrown to
            // the outputs, and the done flag can be raised so the ALU
            // knows the operation has concluded.
            end else begin
                {cout, Y}   = result;
                //Since the output of a multiplier is 2x number of
                // input bits and our output is staying at 8 bits
                // the overflow flag needs to be raised whenever
                // the result exceeds 8 bits

                if(cout > 0) begin
                    overflow <= 1;
                end else begin
                    overflow <= 0;
                end

                //Raising the done flag should always be the last
                // thing it does
                done        <= 1'b1;
            end
        end
    end

endmodule


```

2.2 ALU Utility Modules

Three SystemVerilog modules were developed to be used within ALU operations or the ALU itself. These include a variable width asynchronously resettable register with enable, a 3 bit multiplexer with enable, and a variable width counter. The Implementation details and SystemVerilog listings for these modules are provided in the subsequent sections.

2.2.1 Opcode Decoder

The decoder takes a 3-bit opcode input and converts it to an 8-bit, one-hot signal used to enable a given operation module. When enable is low, all outputs are set to 0.

Listing: multiplexer_3_en.sv

```
/*
Project: 8-Bit ALU
Author: Nolan Kessler

Implements a three bit multiplexer with enable
*/

module Decoder (
    input logic [2 : 0] in,
    input logic enable, //Output of multiplexer is 0 unless enable is high.
    output logic [7 : 0] out
);
    always_comb begin
        if (enable) begin
            case (in)
                3'b000: out = 8'b0000_0001;
                3'b001: out = 8'b0000_0010;
                3'b010: out = 8'b0000_0100;
                3'b011: out = 8'b0000_1000;
                3'b100: out = 8'b0001_0000;
                3'b101: out = 8'b0010_0000;
                3'b110: out = 8'b0100_0000;
                3'b111: out = 8'b1000_0000;
            endcase
        end
        else begin
            out = 4'b0;
        end
    end
endmodule
```

2.2.2 Variable Width Register

The variable width register is a configurable register, with a default width of 1, used to store the value of the in signal. If enable is high, the input "in" is copied to the output on the rising clock edge. Then, when enable is low, the register holds the value of out. If reset goes low, the output is immediately set to 0.

Listing: decoder.sv

```
/*
Project: 8-Bit ALU
Author: Nolan Kessler
Sources: Based on my register implementation from Lab 5

Implements a parameterized width register with enable and active low reset.
*/

module Register_en_rstn #(parameter WIDTH = 1) (
    input logic clk,
    input logic enable, //Enable - copy in to out on rising edge of clk only when high
    input logic rst_n, //Active low reset - set out to zero on falling edge
    input logic [WIDTH - 1 : 0] in,
    output logic [WIDTH - 1 : 0] out
);

    always_ff @(posedge clk, negedge rst_n) begin

        if (rst_n == 0) begin
            out <= 0;
        end

        else begin
            if (enable != 0) begin
                out <= in;
            end
        end

    end

endmodule
```

2.2.3 Counter

The counter is an N width counting module with a default width of 4 bits. On the rising clock edge, the counter increments itself by the value of addBy. If reset is high, the counter resets the count to 0.

Listing: counter.sv

```
/*
Project: 8 Bit ALU
Author: Nolan Kessler
Sources: Based on my counter implementation from lab 5

A configurable counter
*/

module Counter
#(
```

```
parameter IN_WIDTH = 4,
parameter OUT_WIDTH = 4
)
(
        input logic clock,
        input logic reset_n,
        input logic enable,
        input logic [IN_WIDTH-1:0] addBy,
        output logic [OUT_WIDTH-1:0] count
);

        logic [OUT_WIDTH-1:0] count_next;

        always_comb begin
                count_next = count + (addBy & {IN_WIDTH{enable}});
        end

        Register_en_rstn #(OUT_WIDTH) register(
                .clk(clock),
                .rst_n(reset_n),
                .in(count_next),
                .out(count),
        .enable(1'b1)
        );

endmodule
```

2.3 ALU Implementation

A schematic diagram of the full ALU is shown in figure 7. Four registers are used to ensure that the inputs to the ALU remain consistent throughout the entire operation. R_a, R_b, and R_op will store the inputs to the ALU on the first rising clock edge that occurs while EN is high. The output of the locking register R_lk becomes high on the next clock falling edge, disabling further updates to A, B, and OP, and enabling the decoder, which will select the requested function. At this point the value of EN can be changed without impacting the ALU operation.

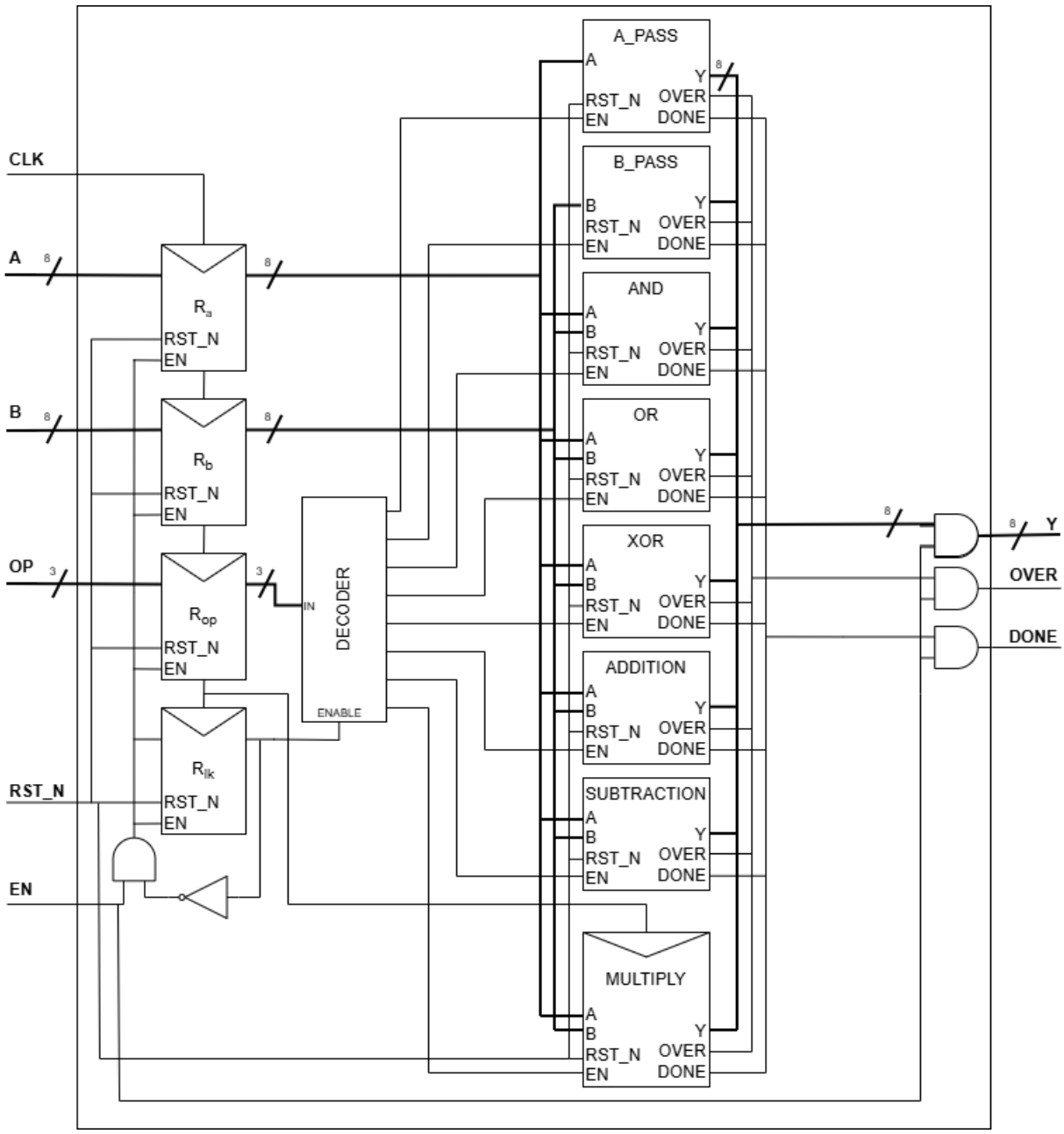


Figure 7: Schematic of Full ALU

3 Project Reflection

Implementing the project overall when well. Implementing each individual combinational modules and getting them working in the larger ALU module was quick and relatively simple. However we ran into some complications when attempting to implement a more complicated sequential module. This was the multiplication module. Since it required multiple clock cycles to perform, the module needed to have proper timing and a comparator to track the current clock cycle the system is on and run the system accordingly. We ran into a few issues with the done and overflow registers being out of sync causing the module to fail to properly output the overflow flag. These smaller difficulties were more time consuming than difficult as they were small errors that required

looking over the HDL script and RTL Viewer to identify where the timing issues would occur.

4 Design Verification

Design verification is done using SystemVerilog test benches. Each operation module has it own test bench to test expected values compared to simulated. The ALU itself its own test bench which ensures the ALU design and utility modules function as expected.

4.1 Operation Test Benches

Each operation module has a test bench. All test benches with the exception of the multiplication test bench test all possible combinations of operands as well as testing the case where the enable signal is low.

4.1.1 A Pass

Listing: a_pass.tb.sv

```
/*
Project: 8-Bit ALU
Author: John O'Connor
Sources: N/A

Testbench file for the A passthorugh module (A_pass.sv).
*/

module A_pass_tb;
        // Inputs
        logic [7:0] A;
        logic enable;
        // Outputs
        logic [7:0] Y;
        logic done;
        logic overflow;

        A_pass dut(
                .A(A),
                .enable(enable),
                .Y(Y),
                .done(done),
                .overflow(overflow)
        );

        initial begin

                // Test case where enable is low
                enable = 0;

                A = 8'b0;
                #10;
                // Y, done, and overflow should all be floating
                if (Y !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
```

```
                        $display("Test Failed enable low case");
                        $stop;
                end

                // Test cases where enable is high
                 enable = 1;

                // Test all possible values for A
                // Expected: Y = A for every test
                for (int i=0; i<256; i++) begin
                        A = i;
                        #10;
                        if (Y !== A || done !== 1'b1 || overflow !== 1'b0) begin
                                $display("Test Failed for A = %0d", i);
                                $stop;
                        end
                end

                $display("All Tests Passed");

        end

endmodule
```

4.1.2 B Pass

Listing: b_pass.tb.sv

```
/*
Project: 8-Bit ALU
Author: John O'Connor
Sources: N/A

Testbench file for the B passthrough module (b_pass.sv).
*/

module B_pass_tb;
        // Inputs
        logic [7:0] B;
        logic enable;
        // Outputs
        logic [7:0] Y;
        logic done;
        logic overflow;

        B_pass dut(
                .B(B),
                .enable(enable),
                .Y(Y),
                .done(done),
                .overflow(overflow)
        );

```

```
        initial begin

                // Test case where enable is low
                enable = 0;

                B = 8'b0;
                #10;
                // Y, done, and overflow should all be floating
                if (Y !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
                        $display("Test Failed enable low case");
                        $stop;
                end

                // Test cases where enable is high
                enable = 1;

                // Test every possible value for B
                // Expected: Y = B for each value
                for (int i=0; i<256; i++) begin
                        B = i;
                        #10;
                        // Check Y is correct, done is high, overflow is low
                        if (Y !== B || done !== 1'b1 || overflow !== 1'b0) begin
                                $display("Test Failed for B = %0d", i);
                                $stop;
                        end
                end

                $display("All Tests Passed");

        end

endmodule
```

4.1.3 Bitwise AND

Listing: and.tb.sv

```
/*
Project: 8-Bit ALU
Author: John O'Connor
Sources: N/A

Testbench file for the And module (bw_and.sv).
*/

module And_tb;
        // Inputs
        logic [7:0] A, B;
        logic enable;
        // Outputs
        logic [7:0] Y;
        logic done;
```

```
        logic overflow;

        Bitwise_AND dut(
                .A(A),
                .B(B),
                .enable(enable),
                .Y(Y),
                .done(done),
                .overflow(overflow)
        );

        initial begin

                // Test case where enable is low
                enable = 0;
                A = 8'b0;
                B = 8'b1;
                #10;
                // Y, done, and overflow should all be floating
                if (Y !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
                        $display("Test Failed enable low case");
                        $stop;
                end

                // Test cases where enable is high
                enable = 1;

                // Test all possible values for A and B
                for(int i=0; i<256; i++) begin
                        for (int j=0; j<256; j++) begin
                                A = i;
                                B = j;
                                #10;
                                // Check Y is correct, done is high, and overflow is low
                                if (Y!== (A&B) || done !== 1'b1 || overflow !== 1'b0) begin
                                        $display("Test Failed");
                                        $stop;
                                end
                        end
                end

                $display("All Tests Passed");
        end

endmodule
```

4.1.4 Bitwise OR

Listing: bw_or.tb.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
```

```
Sources: N/A

Testbench file for the OR module (bw_or.sv).
*/

module Or_tb();

    //Module Inputs
    logic [7:0] A_input, B_input;
    logic enable;

    //Module Outputs
    logic [7:0] Y_test;
    logic done, overflow;

    //Module Call
    Bitwise_OR dut(
        .A(A_input), .B(B_input),
        .enable(enable),
        .Y(Y_test),
        .overflow(overflow), .done(done)
    );

    initial begin

        // Test case where enable is low
        enable = 0;
        A_input = 8'b0;
        B_input = 8'b1;
        #10;
        // Y, done, and overflow should all be floating
        if (Y_test !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
            $display("Test Failed enable low case");
            $stop;
        end

        // Test cases where enable is high
        enable = 1;

        // Test all possible values for A and B
        for(int A = 0; A < 256; A++) begin
            for(int B = 0; B < 256; B++) begin
                A_input = A; B_input = B;
                #10;
                // Check Y is correct, done is high, and overflow is low
                if (Y_test !== (A_input | B_input) || done !== 1'b1 || overflow !== 1'b0)
                ↪  begin
                    $display("Failed at %dA | %dB; Y=%d", A, B, Y_test);
                    $stop;
                end
            end
        end

        $display("All Tests Passed");
    end

```

```
endmodule
```

4.1.5 Bitwise XOR

Listing: bw_xor.tb.sv

```
/*
Project: 8-Bit ALU
Author: John O'Connor
Sources: N/A

Testbench file for the Bitwise XOR (bw_xor.sv).
*/

module Xor_tb;
        // Inputs
        logic [7:0] A, B;
        logic enable;
        // Outputs
        logic [7:0] Y;
        logic done;
        logic overflow;

        Bitwise_XOR dut(
                .A(A),
                .B(B),
                .enable(enable),
                .Y(Y),
                .done(done),
                .overflow(overflow)
        );

        initial begin

                // Test case where enable is lwo
                enable = 0;

                A = 8'b0;
                B = 8'b1;
                #10;
                // Y, done, and overflow should all be floating
                if (Y !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
                        $display("Test Failed enable low case");
                        $stop;
                end

                // Test cases where enbale is high
                enable = 1;

                // Test every possible combination of A and B
                for(int i=0; i<256; i++) begin
                        for (int j=0; j<256; j++) begin
                                A = i;
```

```
                                B = j;
                                #10;
                                // Check Y is correct, done is high, and overflow is low
                                if (Y!== (A^B) || done !== 1'b1 || overflow !== 1'b0) begin
                                        $display("Test Failed");
                                        $stop;
                                end
                        end
                end

                $display("All Tests Passed");
        end

endmodule
```

4.1.6 Addition

Listing: addition.tb.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian

Addition Testbench, tests all possible outputs of adding two 8-bit busses
*/
module AdditionTestbench();

//Module Inputs
logic [7:0] A_input, B_input;
logic enable;

//Module Outputs
logic [7:0] Y_test;
logic done, overflow;

//Expected Outputs
logic [7:0] Y_exp;
logic overflow_exp;

Addition dut(
    .A(A_input), .B(B_input),
    .enable(enable),
    .Y(Y_test),
    .overflow(overflow), .done(done)
);

initial begin

    // Test case where enable is low
    enable = 0;
    A_input = 8'b0;
    B_input = 8'b1;
    #10;
```

```
    // Y, done, and overflow should all be floating
    if (Y_test !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
        $display("Test Failed enable low case");
        $stop;
    end

    // Test cases where enable is high
    enable = 1;

    // Test all possible values for A and B
    for(int A = 0; A < 256; A++) begin
        for(int B = 0; B < 256; B++) begin
            A_input = A; B_input = B;
            {overflow_exp, Y_exp} = A + B;
            #10;
            // Check Y is correct, done is high, and overflow is correct
            if (Y_test !== Y_exp || done !== 1'b1 || overflow !== overflow_exp) begin
                $display("Failed at %dA + %dB; Y=%d and Y_exp=%d and overflow=%d and
                ↪  overflow_exp=%d",
                    A, B, Y_test, Y_exp, overflow, overflow_exp);
                $stop;
            end
        end
    end

    $display("All Tests Passed");
end

endmodule
```

4.1.7 Subtraction

Listing: subtraction.tb.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian

Subtraction Testbench, tests all possible outputs of subtracting two 8-bit busses
*/
module SubtractionTestbench();

//Module Inputs
logic [7:0] A_input, B_input;
logic enable;

//Module Outputs
logic [7:0] Y_test;
logic done, overflow;

//Expected Outputs
logic [7:0] Y_exp;

```

```
//Module Call
Subtraction dut(
    .A(A_input), .B(B_input),
    .enable(enable),
    .Y(Y_test),
    .overflow(overflow), .done(done)
);

initial begin

    // Test case where enable is low
    enable = 0;
    A_input = 8'b0;
    B_input = 8'b1;
    #10;
    // Y, done, and overflow should all be floating
    if (Y_test !== 8'bz || done !== 1'bz || overflow !== 1'bz) begin
        $display("Test Failed enable low case");
        $stop;
    end

    // Test cases where enable is high
    enable = 1;

    // Test all possible values for A and B
    for(int A = 0; A < 256; A++) begin
        for(int B = 0; B < 256; B++) begin
            A_input = A; B_input = B;
            Y_exp = A - B;
            #10;
            // Check Y is correct, done is high, overflow is always 0 for subtraction
            if (Y_test !== Y_exp || done !== 1'b1 || overflow !== 1'b0) begin
                $display("Failed at %dA - %dB; Y=%d and Y_exp=%d",
                    A, B, Y_test, Y_exp);
                $stop;
            end
        end
    end

    $display("All Tests Passed");
end

endmodule
```

4.1.8 Multiplication

Listing: mult.tb.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian
Sources:
    *https://en.wikipedia.org/wiki/Binary_multiplier#Single-cycle_multiplier
```

```
6
    ↪ *https://stackoverflow.com/questions/63655634/8-bit-sequential-multiplier-using-add-and-shift
7
8   Sequential Multiplier Test Bench. Tests one special case, and all possible outputs in
    ↪ multiplying 2 6-bit busses
9   */
10  module Mult_8bitTestbench();
11
12  //Module Inputs
13  logic [7:0] A_input, B_input;
14  logic enable, clock, reset_n;
15
16  //Module Outputs
17  logic [7:0] Y_test;
18  logic cout_test, Done_test;
19
20  //Expected Outputs
21  logic [7:0] Y_exp;
22  logic cout_exp, Done_exp;
23
24  //Additional Logic to Help account for overflow
25  logic [7:0] overflow_catch;
26
27  //Module Call
28  Mult_8bit dut(
29      .A(A_input), .B(B_input),
30      .enable(enable), .clock(clock), .reset_n(reset_n),
31      .Y(Y_test),
32      .overflow(cout_test), .done(Done_test)
33  );
34
35
36  task reset();
37  /*
38  * Reset Task:
39      * Runs the reset pin at active low
40      * Should reset all registers to 0 and set done flag to 0
41      * And sets outputs to floating
42  */
43      reset_n = 1'b0;
44      #10;
45      reset_n = 1'b1;
46  endtask
47
48  task multiply();
49  /*
50  * Multiply Task:
51      * Since the multiplier Module runs on a number of clock cycles
52      * The test case needs to be predictable for proper comparison
53      *
54      * Therefore the multiply function runs the clock until the Done flag is
55      *  raised. Allowing thee multiplier enough time to complete the
56      *  multiplication.
57  */
58      while (Done_test !== 1'b1) begin
59          clock = 1'b1;
```

```
        #1;
        clock = 1'b0;
        #1;
    end
endtask

task validate();
/*
* Validate Task:
    * Iterates through a limited set of values for A and B
    *  and compares the calculated values of multiplication
    *  to the implimented addition module
    *
    * Since the output is limited to 8-bits the multiplier
    *  technically only works for two 4-bit inputs,
    *  and the each test takes 9 clock cycles
    *  so the test is limited to a 6-bit maximum
*/

    int A2; //Declare variables for use in later test case
    int B2;

    for(int A = 0; A < 64; A++) begin
        //If enable is off we should see a floating output
        if(enable == 0) begin
            //Running multiply here causes the module to run forever
            // Which proves that it works correctly
            // However this is also why it is commented out.

            //multiply();
            $display("Function not yet enabled %dY_test %dcout_test",
                Y_test, cout_test);
            break;
        end
        for(int B = 0; B < 64; B++) begin
            //Assigns the module bus inputs to current values
            A_input = A; B_input = B;
            //Calculates expected values and outputs them to expected
            {overflow_catch, Y_exp} = A * B;
            //Overflow Catch exists because of the output size limitation
            // always overflows when the output is bigger than the output
            // instead of only when the 9th bit is 1 like for other TBs
            if(overflow_catch > 0) begin cout_exp = 1; end

            multiply(); //Run the multiply task to give enough clock cycles
            #5;         //Pause to let module process

            //Checks all expected to tested from module, shows where it fails
            if(cout_exp !== cout_test && Y_exp !== Y_test && Done_test !== Done_exp) begin
                $display("Failed at %dA * %dB; Y=%d and Y_exp=%d and cout=%d and cout_exp=%d",
                    A, B, Y_test, Y_exp, cout_test, cout_exp);
                //Does not stop at fail to identify the pattern of failure
                // from test results
            end
            reset();
        end
```

```
    end
    if(enable == 1) begin
        //This is a special case test for when the value should be
        // massively large but will appear much smaller due to the limitation
        // of the output.
        //
        //Same as the looped cases above
        A2 = 255;
        B2 = 128;
        A_input = A2;
        B_input = B2;
        {cout_exp, Y_exp} = A2 * B2;

        multiply();
        #5;

        if(cout_exp !== cout_test && Y_exp !== Y_test && Done_test !== Done_exp) begin
            $display("Failed at %dA * %dB; Y=%d and Y_exp=%d and cout=%d and cout_exp=%d",
                A2, B2, Y_test, Y_exp, cout_test, cout_exp);
            //$stop;
        end
        reset();
    end
endtask

initial begin
    //Initialize Variables
    enable  = 1'b0; //First test enable at 0
    A_input = 0;
    B_input = 0;
    Done_exp= 1;    //Expected Done flag will always be 1

    reset();        //Reset to begin, otherwise last ran test will be output
    validate();     //Should display the not yet enabled message

    enable = 1'b1;  //Now test enable at 1

    reset();        //Reset for same reason as before
    validate();     //Should pass with no error

    #100;           //Take a breather
    $stop;
end

endmodule
```

4.2 ALU Test Bench

The ALU test bench tests the ALU design as well as the functionality of the utility modules. It iterates through each operation by testing one opcode at a time, resetting between tests.

Listing: alu.tb.sv

```
module ALU_tb;
    //inputs
    logic        en;
    logic        clk;
    logic        rst_n;
    logic [7 : 0]  a, b;
    logic [2 : 0]  op;
    //outputs
    logic [7:0]  y;
    logic        overflow;
    logic        done;

    alu dut (
        .en(en),
        .clk(clk),
        .rst_n(rst_n),
        .a(a),
        .b(b),
        .op(op),
        .y(y),
        .overflow(overflow),
        .done(done)
    );

    always #5 clk = ~clk;

    // task to reset between operations
    task reset();
        en    = 0;
        rst_n = 0;
        #10;
        rst_n = 1;
    endtask

    // executing task
    task execute();
        @(negedge clk);
        en = 1;
        #5;
        @(posedge done);
        #5;
    endtask

    initial begin
        clk = 0;
        a   = 0;
        b   = 0;
        op  = 0;
        en  = 0;
        reset();

        // a_pass (0x0)
        reset();
```

```
        a = 8'hAA; b = 8'h55; op = 4'h0;
        execute();
        if (y !== 8'hAA || done !== 1'b1 || overflow !== 1'b0) begin
            $display("FAILED: A_pass");
            $stop;
        end

        // B_pass (0x1)
        reset();
        a = 8'hAA; b = 8'h55; op = 4'h1;
        execute();
        if (y !== 8'h55 || done !== 1'b1 || overflow !== 1'b0) begin
            $display("FAILED: B_pass");
            $stop;
        end

        // Bitwise AND (0x2)
        reset();
        a = 8'hAA; b = 8'h55; op = 4'h2;
        execute();
        if (y !== 8'h00 || done !== 1'b1 || overflow !== 1'b0) begin
            $display("FAILED: Bitwise AND");
            $stop;
        end

        // Bitwise OR (0x3)
        reset();
        a = 8'hAA; b = 8'h55; op = 4'h3;
        execute();
        if (y !== 8'hFF || done !== 1'b1 || overflow !== 1'b0) begin
            $display("FAILED: Bitwise OR");
            $stop;
        end

        // Bitwise XOR (0x4)
        reset();
        a = 8'hAA; b = 8'h55; op = 4'h4;
        execute();
        if (y !== 8'hFF || done !== 1'b1 || overflow !== 1'b0) begin
            $display("FAILED: Bitwise XOR");
            $stop;
        end

        // Addition (0x5)
        reset();
        a = 8'h01; b = 8'h02; op = 4'h5;
        execute();
        if (y !== 8'h03 || done !== 1'b1 || overflow !== 1'b0) begin
            $display("FAILED: Addition");
            $stop;
        end

        // Addition with overflow (0x5)
        reset();
        a = 8'hFF; b = 8'h01; op = 4'h5;
        execute();
```

```
        if (y !== 8'h00 || done !== 1'b1 || overflow !== 1'b1) begin
            $display("FAILED: Addition overflow");
            $stop;
        end

        // Subtraction (0x6)
        reset();
        a = 8'h05; b = 8'h03; op = 4'h6;
        execute();
        if (y !== 8'h02 || done !== 1'b1 || overflow !== 1'b0) begin
            $display("FAILED: Subtraction");
            $stop;
        end

        // Multiplication (0x7)
        reset();
        a = 8'h05; b = 8'h03; op = 4'h7;
        execute();
        if (y !== 8'h0F || done !== 1'b1 || overflow !== 1'b0) begin
            $display("FAILED: Multiplication");
            $stop;
        end

        $display("All Tests Passed!");
        $stop;
    end
endmodule
```

4.3 Simulation Testing

Simulation can be done using ModelSim. To accompany this, we can run a do file which runs all test benches.

Listing: ALUSim.do

```
# Project: 8-Bit ALU
# Author: John O'Connor
# Sources: None

# Do file for running all independent testbenches

#-nostdout
# Run module testbenches
vsim -quiet A_pass_tb -do "run -all"
vsim -quiet B_pass_tb -do "run -all"
vsim -quiet And_tb -do "run -all"
vsim -quiet Or_tb -do "run -all"
vsim -quiet Xor_tb -do "run -all"
vsim -quiet AdditionTestbench -do "run -all"
vsim -quiet SubtractionTestbench -do "run -all"
vsim -quiet Mult_8bitTestbench -do "run -all"

```

```
# ALU testbench
vsim ALU_tb
add wave *
run -all
```

Do File Output

```
# End time: 18:45:52 on Jun 07,2026, Elapsed time: 0:23:45
# Errors: 6, Warnings: 2
# vsim -quiet A_pass_tb -do "run -all"
# Start time: 18:45:52 on Jun 07,2026
# run -all
# All Tests Passed
# Errors: 3, Warnings: 1
# vsim -quiet B_pass_tb -do "run -all"
# Start time: 18:45:55 on Jun 07,2026
# run -all
# All Tests Passed
# End time: 18:45:59 on Jun 07,2026, Elapsed time: 0:00:04
# Errors: 3, Warnings: 1
# vsim -quiet And_tb -do "run -all"
# Start time: 18:45:59 on Jun 07,2026
# run -all
# All Tests Passed
# End time: 18:46:02 on Jun 07,2026, Elapsed time: 0:00:03
# Errors: 3, Warnings: 1
# vsim -quiet Or_tb -do "run -all"
# Start time: 18:46:02 on Jun 07,2026
# run -all
# All Tests Passed
# End time: 18:46:06 on Jun 07,2026, Elapsed time: 0:00:04
# Errors: 3, Warnings: 1
# vsim -quiet Xor_tb -do "run -all"
# Start time: 18:46:06 on Jun 07,2026
# run -all
# All Tests Passed
# End time: 18:46:10 on Jun 07,2026, Elapsed time: 0:00:04
# Errors: 3, Warnings: 1
# vsim -quiet AdditionTestbench -do "run -all"
# Start time: 18:46:10 on Jun 07,2026
# run -all
# All Tests Passed
# Errors: 3, Warnings: 1
# vsim -quiet SubtractionTestbench -do "run -all"
# Start time: 18:46:14 on Jun 07,2026
# run -all
# All Tests Passed
# End time: 18:46:17 on Jun 07,2026, Elapsed time: 0:00:03
# Errors: 3, Warnings: 1
# vsim -quiet Mult_8bitTestbench -do "run -all"
# Start time: 18:46:17 on Jun 07,2026
# run -all
# Function not yet enabled   zY_test zcout_test
```

```
# ** Note: $stop    : C:/Users/jacko/ece204/Project-ALU/ALU_SystemVerilog/
   functions/testbenches/mult.tb.sv(150)
#    Time: 135321 ps  Iteration: 0  Instance: /Mult_8bitTestbench
# Break in Module Mult_8bitTestbench at C:/Users/jacko/ece204/Project-ALU/
   ALU_SystemVerilog/functions/testbenches/mult.tb.sv line 150
# End time: 18:46:21 on Jun 07,2026, Elapsed time: 0:00:04
# Errors: 3, Warnings: 1
# vsim ALU_tb
# Start time: 18:46:21 on Jun 07,2026
# Loading sv_std.std
# Loading work.ALU_tb
# Loading work.alu
# Loading work.Register_en_rstn
# Loading work.Decoder
# Loading work.A_pass
# Loading work.B_pass
# Loading work.Bitwise_AND
# Loading work.Bitwise_OR
# Loading work.Bitwise_XOR
# Loading work.Addition
# Loading work.Subtraction
# Loading work.Mult_8bit
# All Tests Passed!
# ** Note: $stop    : C:/Users/jacko/ece204/Project-ALU/ALU_SystemVerilog/alu.
   tb.sv(134)
#    Time: 360 ps  Iteration: 0  Instance: /ALU_tb
# Break in Module ALU_tb at C:/Users/jacko/ece204/Project-ALU/ALU_SystemVerilog
   /alu.tb.sv line 134
```

ALU Test Bench Waveform

We can analyze the ALU test bench's waveform using the waveform

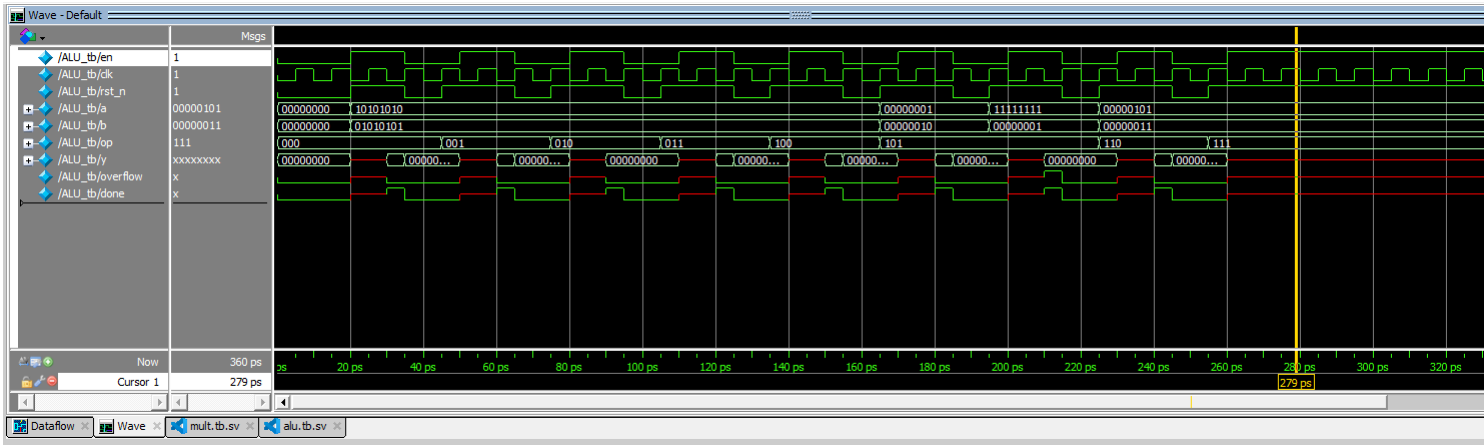


Figure 8: Waveform Generated by ALU Test Bench

5 Hardware Validation

Hardware validation requires a couple additional modules for the ALU to work effectively on the FPGA. To achieve two 8-bit inputs, we need to be able to input operand A, then have a register

hold that value while we input operand B, then a register to hold B while we input the opcode. This is a achieved using a four state FSM, where the state is used to control a four-bit bus demultiplexer. When the final user input state is finished, the ALU is enabled and the state counter is disabled on the next clock falling edge. The display output is switched from the switch input (Num) to the alu output Y, using a one bit bus multiplexer. The FSM will remain in this state until the system is reset. A schematic and state transition diagram are shown in figures **??**ALU Hardware Implementation Schematic) and **??** respectively. Listings of the SystemVerilog implementations of this system and its components are included in the next section.

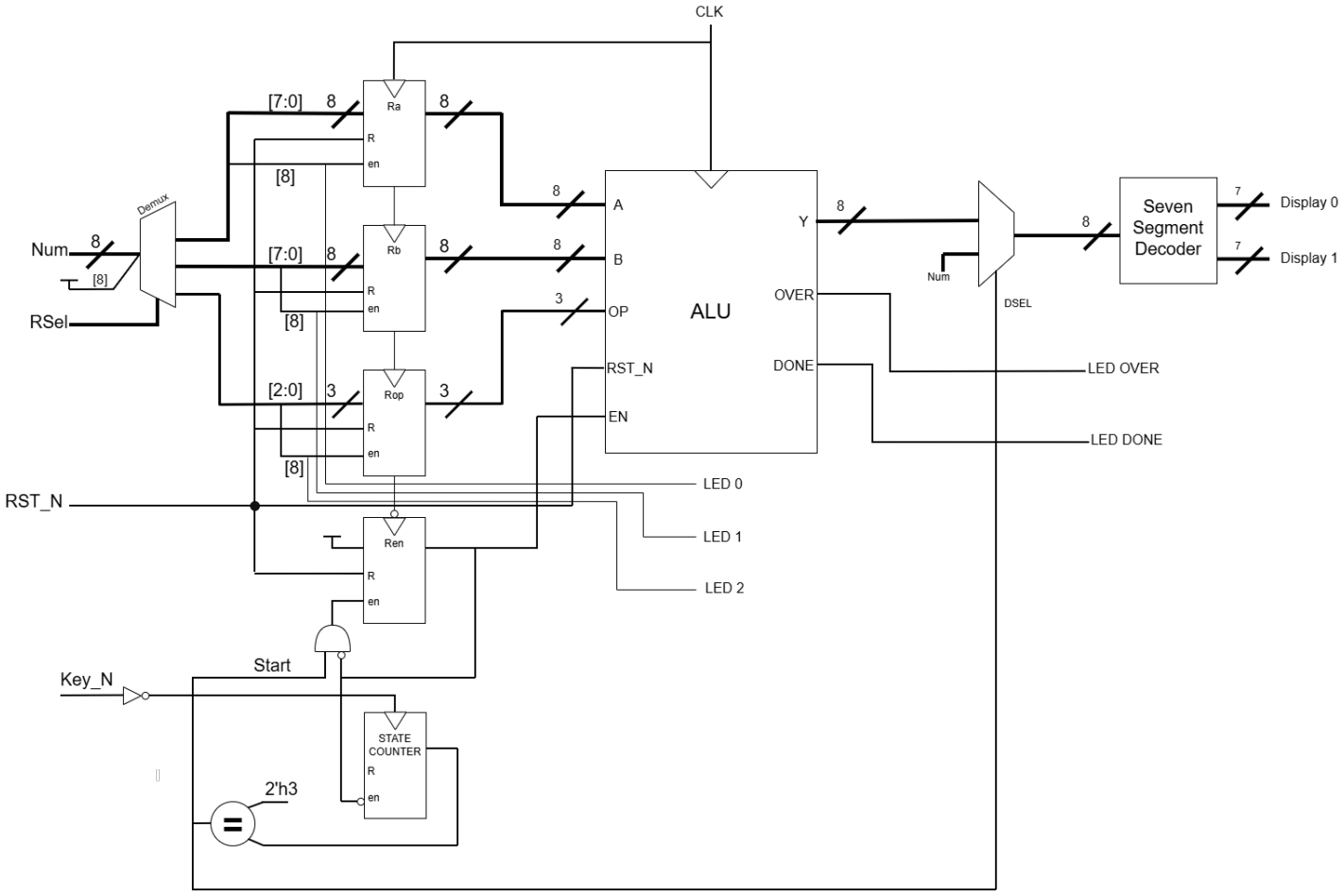


Figure 9: ALU Hardware Implementation Schematic

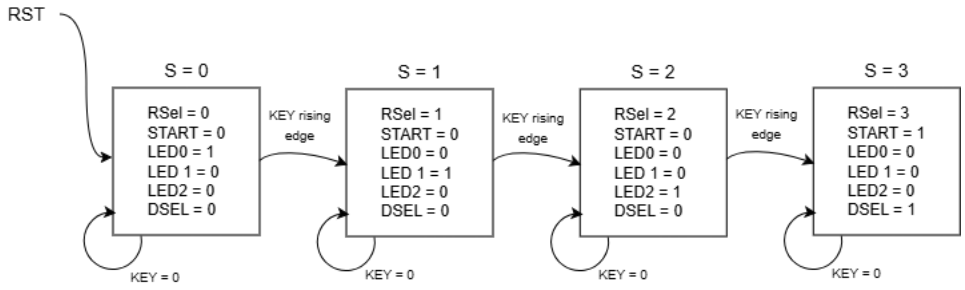


Figure 10: ALU Hardware Implementation State Machine

5.1 Hardware Implementation HDL Listings

Listing: alu_test.sv

```
/*
Project: 8-Bit ALU
Author: Nolan Kessler

Implements interactive testing of ALU on hardware
*/

module alu_test(
    input logic key_n,
    input logic rst_n,
    input logic clk,
    input logic [7:0] switches,
    output logic [2:0] step_leds,
    output logic done_led,
    output logic overflow_led,
    output logic [6:0] seven_seg0,
    output logic [6:0] seven_seg1
);

    logic key;
    logic [1:0] state;
    logic last_step;
    logic running;
    logic [8:0] reg_a_bus;
    logic [8:0] reg_b_bus;
    logic [8:0] reg_op_bus;
    logic [7:0] alu_a;
    logic [7:0] alu_b;
    logic [7:0] alu_op_wide;
    logic [2:0] alu_op;
    logic [7:0] alu_y;
    logic [3:0] digit0;
    logic [3:0] digit1;

    assign key = ~key_n; //Inverted version of button for use with active high modules
    assign last_step = state == 3; //Comparator to track when state machine is on last step, and ALU is running
    assign alu_op = alu_op_wide[2:0];
    assign step_leds[0] = reg_a_bus[8]; //Leds to indicate which step system is on
    assign step_leds[1] = reg_b_bus[8];
    assign step_leds[2] = reg_op_bus[8];

    Counter #(1, 2) state_counter (
        .clock(key),
        .reset_n(rst_n),
        .enable(~last_step),
        .addBy(1'b1),
        .count(state)
    );

    Register_en_rstn #(8) reg_a (
```

```
        .clk(clk),
        .enable(reg_a_bus[8]),
        .rst_n(rst_n),
        .in(reg_a_bus[7:0]),
        .out(alu_a)
    );

    Register_en_rstn #(8) reg_b (
        .clk(clk),
        .enable(reg_b_bus[8]),
        .rst_n(rst_n),
        .in(reg_b_bus[7:0]),
        .out(alu_b)
    );

    Register_en_rstn #(8) reg_op (
        .clk(clk),
        .enable(reg_op_bus[8]),
        .rst_n(rst_n),
        .in(reg_op_bus[7:0]),
        .out(alu_op_wide)
    );

    Register_en_rstn #(1) reg_start (
        .clk(~clk),
        .enable(last_step),
        .rst_n(rst_n),
        .in(1'b1),
        .out(running)
    );

    Bus_Demultiplexer_2 #(9) reg_selector (
        .in({1'b1, switches}),
        .sel(state),
        .out0(reg_a_bus),
        .out1(reg_b_bus),
        .out2(reg_op_bus)
    );

    Bus_Multiplexer_1 #(8) disp_source_selector (
        .in0(switches),
        .in1(alu_y),
        .sel(last_step),
        .out({digit1, digit0})
    );

    SevenSegmentDecode disp0 (
        .digit(digit0),
        .segments(seven_seg0)
    );

    SevenSegmentDecode disp1 (
        .digit(digit1),
        .segments(seven_seg1)
    );

```

```
    alu ALU (
        .en(running),
        .clk(clk),
        .rst_n(rst_n),
        .a(alu_a),
        .b(alu_b),
        .op(alu_op),
        .y(alu_y),
        .overflow(overflow_led),
        .done(done_led)
    );



endmodule
```

Listing: bus_demultiplexer_2.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian

Bus 1:4 Reverse Multiplexer, Takes one N-bit Bus and selector passes said bus to specific output
*/
module Bus_Demultiplexer_2 #(parameter BUS_WIDTH = 4) ( //Parameter to choose bus Width with N number of bits
    input logic [BUS_WIDTH - 1 : 0] in,    //N-bit bus input
    input logic [1:0] sel,                 //2-bit selector
    output logic [BUS_WIDTH - 1 : 0] out0, //N-bit bus output
    output logic [BUS_WIDTH - 1 : 0] out1, //N-bit bus output
    output logic [BUS_WIDTH - 1 : 0] out2, //N-bit bus output
    output logic [BUS_WIDTH - 1 : 0] out3  //N-bit bus output
);

    //All outputs will pass 0 when not selected
    assign out0 = sel[1] ? 0: (sel[0] ? 0 : in); //out0 passes in when selector is 00
    assign out1 = sel[1] ? 0: (sel[0] ? in : 0); //out1 passes in when selector is 01
    assign out2 = sel[1] ? (sel[0] ? 0 : in) :0; //out2 passes in when selector is 10
    assign out3 = sel[1] ? (sel[0] ? in : 0) :0; //out3 passes in when selector is 11

endmodule
```

Listing: abus_multiplexer_1.sv

```
/*
Project: 8-Bit ALU
Author: Max Fabian

Bus 2:1 Multiplexer, 2 N-bit busses, with capability to select one of them.
*/
```

```
module Bus_Multiplexer_1 #(parameter BUS_WIDTH) ( //Parameter for choosing Width of N number of bits
    input logic [BUS_WIDTH - 1 : 0] in0, //N-bit bus input
    input logic [BUS_WIDTH - 1 : 0] in1, //N-bit bus input
    input logic sel,                     //1-bit selector
    output logic [BUS_WIDTH - 1 : 0] out //N-bit bus output
);

    assign out = sel ? in1 : in0; //Out is selected dependant on selector

endmodule
```

Listing: SevenSegmentDecode.sv

```
/*
 Project: 8 Bit ALU
 Seven segment decoder implementation from lab 3

 Nolan Kessler
*/

/*
 * ECE 272 Lab 3b
 * Seven Segment Decoder Skeleton File
 *
 * Adapted from Harris & Harris "Digital Design and Computer Achitecture"
 * Example 4.25 "Seven-Segment Display Decoder" (Page 198)
 *
 * Author(s): Quinn Yockey
 * Last Modified: 03 November 2025
 */

/* IMPORTANT!  Read the comments carefully.  They are included to help you!  */

/*
 * Instructions:
 *     Tasks are marked with a number in brackets (e.g., [1]). Each has a
 *     corresponding TODO comment to replace with SystemVerilog code.
 */

/*
 * Declare a module named `SevenSegmentDecode` to convert a 4-bit hexadecimal
 * number to a 7-bit binary value to output on a 7 segment display.
 *
 * [1] Add a 7-bit logic output named `segments` to the `SevenSegmentDecode`
 *     module.
 */
module SevenSegmentDecode(
    input logic [3:0] digit,
    output logic [6:0] segments
);

/*
```

```
 * Using the `always_comb` keyword, create a block that only allows
 * combinational logic.  This is necessary for the `case` statement used within.
 *
 * Like a `switch` statement in C and other programming languages, the `case`
 * statement in SystemVerilog matches the value of the argument to determine
 * what happens next.  In the given branch, if the value of `digit` is 0x0, then
 * use blocking assignment (`=`) to set the value of `segments` to 0b100_0000.
 *
 * Remember that the seven segment displays on the DE10-lite FPGA are active
 * low, so a segment set to 1 will be off and a 0 will be on.  Also, the
 * `segments` has segment a as bit 0 (rightmost) and segment g as bit
 * 6 (leftmost).
 *
 * Refresher on numerical prefixes in SystemVerilog:
 * - 4'h: number is 4 bits wide and formatted in hexadecimal.
 *       (e.g.: 4'h5 = 0x5 = 0b0101 = 5; 4'hC = 0xC = 0b1100 = 12)
 * - 7'b: number is 7 bits wide and formatted in binary.
 *
 * [2] Add branches for the remaining values from 0x1 to 0xF using the values
 *     you listed in figure 3.3 in Lab 3.
 */
always_comb begin
    case (digit)
        //                 abc_defg
        4'h0: segments = 7'b000_0001;
                4'h1: segments = 7'b100_1111;
                4'h2: segments = 7'b001_0010;
                4'h3: segments = 7'b000_0110;
                4'h4: segments = 7'b100_1100;
                4'h5: segments = 7'b010_0100;
                4'h6: segments = 7'b010_0000;
                4'h7: segments = 7'b000_1111;
                4'h8: segments = 7'b000_0000;
                4'h9: segments = 7'b000_1100;
                4'hA: segments = 7'b000_1000;
                4'hB: segments = 7'b110_0000;
                4'hC: segments = 7'b011_0001;
                4'hD: segments = 7'b100_0010;
                4'hE: segments = 7'b011_0000;
                4'hF: segments = 7'b011_1000;
    endcase
end

endmodule
```

6 Reflection on Design Process

Implementing the project overall when well. Implementing each individual combinational modules and getting them working in the larger ALU module was quick and relatively simple. However we ran into some complications when attempting to implement a more complicated sequential module. This was the multiplication module. Since it required multiple clock cycles to perform, the module needed to have proper timing and a comparator to track the current clock cycle the system is on and run the system accordingly. We ran into a few issues with the done and overflow registers being out of sync causing the module to fail to properly output the overflow flag. These smaller difficulties were more time consuming than difficult as they were small errors that required looking over the HDL script and RTL Viewer to identify where the timing issues would occur.